Quantized, Sparsity Aware CNN Accelerator with AXI4-Lite Integration and Clock Domain Crossing

Design of a CNN Accelerator Subsystem with AXI4 Interface

Internal Hackathon, Hardware and VLSI Division

Outline

1 Problem and Objective
2 System Architecture
3 CNN Design
4 AXI4-Lite Interface
5 Clock Domain Crossing
6 Zero Gating and Sparsity
7 Verification and Results
8 Trade offs and Outlook

Problem statement

- The task asks for a complete accelerator subsystem, not an isolated convolution block
- Required parts: a CNN compute architecture, an AXI based interface to a processor, and clock domain crossing between two different clocks
- At least one simple power or logic optimization is required, with zero gating as the suggested technique
- Expected structure: AXI interface, control and data logic, CNN accelerator, output
- Evaluation favors architecture, design understanding and trade offs over RTL completeness

Objective of this work

Design, implement and justify a quantized CNN inference subsystem in which every architectural choice, clock crossing and optimization is explained and measurable.

Four coupled design problems

Problem	What was built	Where
CNN compute	Two convolution, two max pooling and two fully connected layers on an INT8 datapath with INT32 accumulation, executed by 8 lane output stationary MAC arrays	conv_layer_engine, fc_layer_engine, pool_engine, pe_array, mac_unit
Processor interface	AXI4-Lite slave with registers for start, status, input pixels, performance counters and ten output scores	axi4_lite_slave
Clock domain crossing	Toggle pulse synchronizer for start, two flop synchronizers for status levels, Gray code asynchronous FIFO for pixels	cdc_pulse_sync, cdc_sync_2ff, async_fifo
Optimization	Activation zero gating with operand isolation in every MAC lane, plus counters that record suppressed multiplies	mac_unit, perf_counters

Requirements and what was delivered

Requirement	Delivered	Source files	Status
Task 1, CNN architecture	Two Conv, two pooling and two FC layers, INT8 with INT32 accumulate, 8 lane arrays	conv, fc and pool engines, pe_array, mac_unit	Implemented
Task 2, AXI4 subsystem	AXI4-Lite slave for configuration, input, start, status and result read back	axi4_lite_slave, cnn_top, cnn_core	Implemented
Task 3, clock domain crossing	Toggle pulse sync, two flop level sync, Gray code FIFO, bundled result buses	cdc_pulse_sync, cdc_sync_2ff, async_fifo	Implemented
Task 4, optimization	Activation zero gating with operand isolation and skip counters	mac_unit, perf_counters	Implemented
Optional, verification	Bit exact NumPy model, schedule model, self checking testbench with three tests	golden_model, cycle_model, tb_cnn_top	Implemented
Latency, throughput, memory	Deterministic schedule model and logical storage accounting	cycle_model, pe_sweep	Modeled
Resources and Fmax	Not reported, because no synthesis flow was run	Not applicable	Future work

Three ideas that shape the whole design

One MAC lane, reused

A single MAC lane design serves convolution and fully connected layers, so there is one datapath to get right and to verify.

A deterministic, data independent schedule

The cycle count is exactly predictable, which lets the testbench check hardware counters against closed form values.

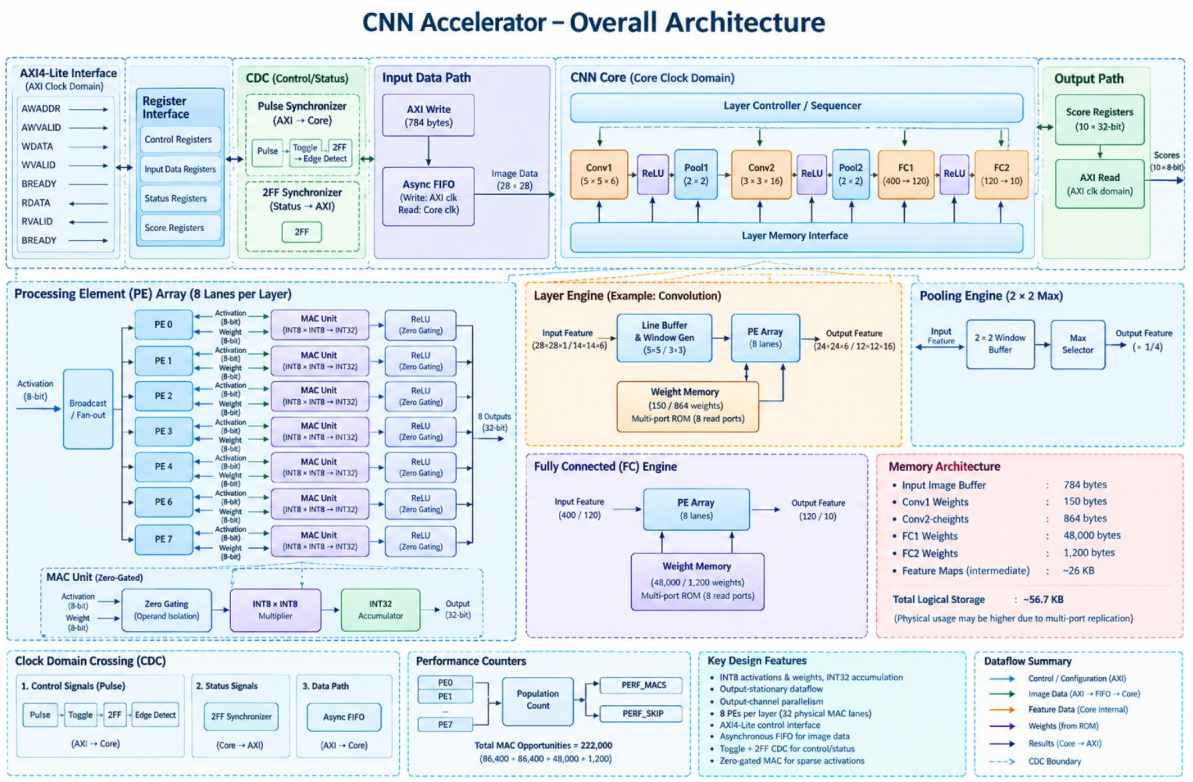
Each CDC technique matched to its signal

A pulse, a slow level and a multi bit stream have different failure modes, so each gets the mechanism that is correct for it.

Results at a glance

Metric	Value
Network	28×28×1 input, Conv 5×5×6, MaxPool, Conv 3×3×16, MaxPool, FC 400 to 120, FC 120 to 10
Precision and parallelism	INT8 activations and weights, INT32 accumulators, 8 MAC lanes per layer engine
Cycles per inference	43,730 from the schedule model
Latency and throughput	291.5 µs at an assumed 150 MHz core clock, about 3,430 inferences per second
MAC lane events	222,000 per inference: 192,184 executed and 29,816 zero gated (13.4%)
Peak gating benefit	FC2 at 59.2% and FC1 at 41.0%
Clock domains	axi_aclk at 100 MHz and core_clk at about 150 MHz, independent
Logical storage	58,086 bytes: weights 50,214 B, feature maps 7,224 B, biases 608 B, scores 40 B

Subsystem architecture



- AXI4-Lite slave runs entirely in axi_aclk
- Three CDC blocks carry start, status and pixels
- The core runs in core_clk: sequencer, memories, four layer engines and two pooling engines
- Scores and counters are read back over the same AXI4-Lite interface
- Flow: AXI slave, CDC, sequencer and engines, scores, AXI read back

Two independent clock domains

- axi_aclk at 100 MHz drives the AXI4-Lite slave and the write side of the input FIFO
- core_clk at about 150 MHz in the testbench drives the sequencer, memories, engines and counters
- Each domain has its own reset: axi_aresetn and core_rst_n
- Four things cross the boundary: the start strobe, three status levels, the pixel stream and the result buses
- cnn_top parameters: AXI_ADDR_W 8, AXI_DATA_W 32, FIFO_ADDR_W 10

Rule

No signal passes between the two domains except through a dedicated CDC block, and the choice of block depends on what the signal carries.

Core sequencer

```
IDLE → LOAD → CONV1 → POOL1 → CONV2 → POOL2 → FC1 → FC2 → DONE
```

- A single FSM in cnn_core.v starts each engine and waits for that engine's done pulse before moving on
- busy is high in every state except IDLE and DONE, and done is high in DONE
- DONE returns directly to LOAD on a fresh start, so back to back inferences need no reset
- In LOAD the sequencer leaves only when the image is fully loaded and no FIFO read is still in flight
- The start pulse is the only control input, and it arrives already synchronized into core_clk

A memory between every pair of stages

Memory	Writer	Reader
img_mem	Input loader	Conv1
conv1_out_mem	Conv1	Pool1
pool1_out_mem	Pool1	Conv2
conv2_out_mem	Conv2	Pool2
pool2_out_mem	Pool2	FC1
fc1_out_mem	FC1	FC2
score registers	FC2	AXI read

- Single port memories, so each has exactly one writer and one reader
- The address port is a mux keyed off the write enable of the producer
- This removes address arbitration entirely
- Every layer boundary can be inspected and verified in isolation
- Flatten is free: channel major addressing already matches the 400 element FC1 input

One inference, end to end

Step	What happens	Modules
1. Push pixels	784 writes to INPUT_DATA become one cycle FIFO writes in axi_aclk	axi4_lite_slave, async_fifo
2. Start	Write 1 to CTRL, the toggle synchronizer delivers one pulse in core_clk, IDLE moves to LOAD	cdc_pulse_sync, cnn_core
3. Load	The loader drains the FIFO into img_mem at one pixel per cycle, counters are cleared	cnn_core
4. Compute	CONV1, POOL1, CONV2, POOL2, FC1 and FC2 each raise done	layer engines
5. Publish	FC2 writes ten INT32 scores, DONE is entered, the done level crosses through two flops	cnn_core, cdc_sync_2ff
6. Read back	The host polls STATUS, then reads PERF_* and the ten scores	axi4_lite_slave

Core cycles: 784 + 17,856 + 4,320 + 12,400 + 2,000 + 6,120 + 250 = 43,730

Network topology and kernel choices

```
Input 28x28x1
→ Conv1 5x5, 6 filters → 24x24x6 → ReLU
→ MaxPool 2x2 → 12x12x6
→ Conv2 3x3, 16 filters → 10x10x16 → ReLU
→ MaxPool 2x2 → 5x5x16 → Flatten 400
→ FC1 400 to 120, ReLU → FC2 120 to 10, raw scores
```

- Two convolution layers: the smallest depth that gives multi channel weight reuse, real feature map storage and a genuine lane sizing problem
- 5×5 first, 3×3 second: the larger receptive field is placed where input channels are fewest

$$N_{\text{taps}} = C_{\text{in}} \times K \times K, \qquad N_{\text{taps}}^{\text{Conv1}} = 25, \qquad N_{\text{taps}}^{\text{Conv2}} = 54$$

A 5×5 kernel in Conv2 would need 150 taps, about 2.8 times more.

Padding, pooling, activation and widths

- Valid padding, stride 1: output size is IH − K + 1, so no artificial zeros and no boundary cases in the address generator
- Max pooling: a compare tree in max_pool2x2, exact in integers, whereas average pooling needs an adder tree, scaling and rounding
- ReLU: one comparator and one mux, against exponentials or lookup tables for sigmoid or tanh. It also creates the zeros that zero gating exploits
- Widths 6, 16, 120, 10: the classic LeNet widths give enough compute to show parallelism with small storage
- Biases: each neuron computes the sum of x·w plus b, one addition per output element, negligible next to hundreds of MACs
- The network is called LeNet inspired, not LeNet-5, because the kernels, pooling and widths are my own choices

Tensor shapes and workload

Stage	Output shape	Elements	Dense MACs
Input	28×28×1	784	0
Conv1, 5×5, 6 filters	24×24×6	3,456	6 × 576 × 25 = 86,400
Pool1, 2×2 max	12×12×6	864	0
Conv2, 3×3, 16 filters	10×10×16	1,600	16 × 100 × 54 = 86,400
Pool2, 2×2 max	5×5×16	400	0
FC1, 400 to 120	120	120	120 × 400 = 48,000
FC2, 120 to 10	10	10	10 × 120 = 1,200
Total			222,000

Output stationary, channel parallel dataflow

Definition

For every output element all of its multiply accumulates happen back to back in one lane's local accumulator, which is written out once at the end.

$$O[c, o_y, o_x] = b_c + \sum_{c_i} \sum_{k_y} \sum_{k_x} X[c_i,\ o_y+k_y,\ o_x+k_x] \cdot W[c,\ c_i,\ k_y,\ k_x]$$

- A partial sum never moves between lanes or through memory
- For a fixed output position every output channel needs the same input window, so one activation is broadcast to all active lanes
- Each lane fetches its own weight for that tap from its own ROM read port
- The fully connected engines use the same structure without the spatial loop

Alternatives that were considered

Dataflow	Stays in the PE	Traffic that grows	Fit for this design
Output stationary	Partial sum	Activation re reads per channel group	Chosen: convolution accumulates locally, control is simple
Weight stationary	Weight	Partial sums move or spill	Good when weight reuse dominates, adds partial sum storage and movement
Input stationary	Activation	Weights and partial sums both move	Highest movement for these kernel sizes

- Cost accepted: the window is re read once per group, twice in Conv2 and 15 times across FC1
- Benefit gained: minimal partial sum traffic and a control structure small enough to verify

Fixed point arithmetic

- Activations and weights are signed INT8, products are 16 bits, accumulators are INT32
- The worst case is FC1 with 400 terms, each of magnitude at most $128 \times 128 = 2^{14}$

$$\mathrm{acc} = \sum_i x_i\, w_i, \qquad x_i,\, w_i \in [-128,\ 127]$$

$$|\mathrm{acc}| \le 400 \times 2^{14} < 2^{23}$$

- INT32 accumulation therefore cannot overflow anywhere, with more than eight bits of headroom
- INT8 gives four times less storage than FP32, small multipliers, high lane density and simple data movement
- FP32 would cost far more area than this problem justifies, and INT16 would double storage and multiplier size without a benefit for a network this small

Requantization between layers

$$y = \mathrm{clip}(\max(0,\ \mathrm{acc} + b) \gg s,\ \ 0,\ \ 127)$$

Layer	Shift s	Output
Conv1	4	INT8 activation
Conv2	6	INT8 activation
FC1	6	INT8 activation
FC2	none	Raw INT32 score

- A fixed right shift replaces a divider with a comparator, a mux and wiring (relu_quant.v)
- ReLU has already made the value non negative, so saturation only clips the upper bound
- The same constants live in cnn_pkg.vh and model/config.py, which keeps RTL and Python bit exact
- Shifts were chosen empirically for the fixed seed weights and must be retuned for a trained model

The MAC lane

- mac_unit.v holds a 32 bit accumulator and one multiplier per lane
- A synchronous acc_clear starts a new output and folds in a simultaneous first product, so no cycle is wasted on a reset
- mac_valid and mac_skipped expose every event so the counters can observe them
- pe_array.v instantiates NUM_PE lanes with a generate loop

```
if (acc_clear) acc <= do_mac ? product : 0;
else if (do_mac) acc <= acc + product;
else acc <= acc; // zero gated cycle: hold
```

The same lane design is instantiated in all four layer engines: 32 physical lanes, one array active at a time.

Four engines, one lane design

- Two convolution engines and two fully connected engines, each with its own 8 lane array
- Chosen over one time multiplexed array because each layer has different address generation
- Separate engines keep control simple and make each layer independently verifiable

Trade off stated honestly

Four arrays of 8 lanes give 32 lanes in total with only one active at a time. A shared array would use a quarter of the lanes, at the cost of an arbiter across four controllers and a more complex address path.

- Listed under future work as the time multiplexed shared array

Layer schedule and cycle model

Stage	Schedule	Cycles
Input load	784 pixels	784
Conv1	576 × (25 + 6)	17,856
Pool1	6 × 144 × 5	4,320
Conv2	2 × 100 × (54 + 8)	12,400
Pool2	16 × 25 × 5	2,000
FC1	15 × (400 + 8)	6,120
FC2	(120 + 8) + (120 + 2)	250
Total		43,730

$$C_{\text{layer}} = \sum_{g} \left(P \cdot N_{\text{taps}} + P \cdot a_g \right)$$

- P = OH × OW for convolution and 1 for FC
- The first term is the tap loop, the second the channel writeback over a_g active lanes
- Pooling costs 5 cycles per output pixel
- The schedule is data independent

Convolution engine control and addressing

```
in_addr = cin*(IN_H*IN_W)
+ (oy+ky)*IN_W + (ox+kx)
wgt_addr[p] = (group*NUM_PE+p)*NTAPS + t
bias_addr[p] = group*NUM_PE + p
en[p] = running && p < active_count
clr[p] = en[p] && t == 0
out_addr = (group*NUM_PE+w)*(OH*OW)
+ oy*OW + ox
```

- Loop nest: group, then oy and ox, then taps over cin, ky and kx, then writeback
- active_count is NUM_PE or COUT minus group × NUM_PE, so the last group can be partial
- Idle lanes stay disabled, which is why Conv1 uses 6 lanes and FC2 ends on 2
- One conv_layer_engine serves Conv1 and Conv2 through parameters

Fully connected and pooling engines

fc_layer_engine

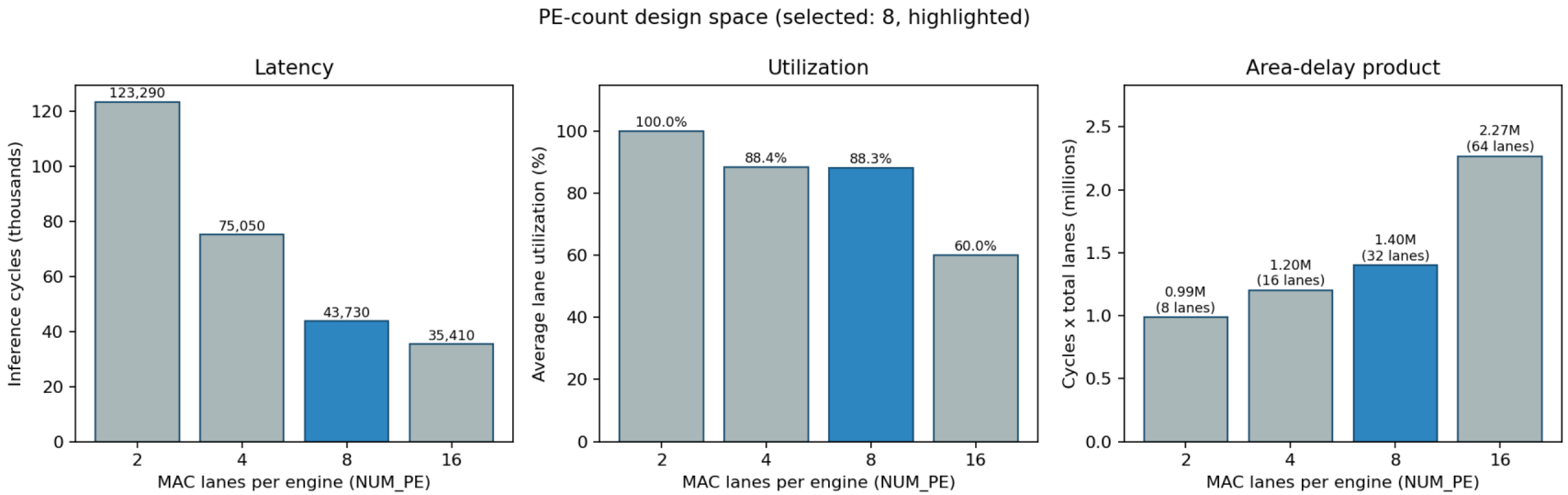
- Same lane structure, no spatial loop
- in_addr = t, weight address (group × 8 + p) × N_IN + t
- FC1: 15 groups, FC2: a group of 8 and a group of 2
- Writes an INT8 activation and a raw INT32 score

pool_engine

- Five cycles per output pixel: four reads and one write
- Reads from a single read port memory
- Pooling is 14.5% of total runtime
- A plain FSM that is easy to verify

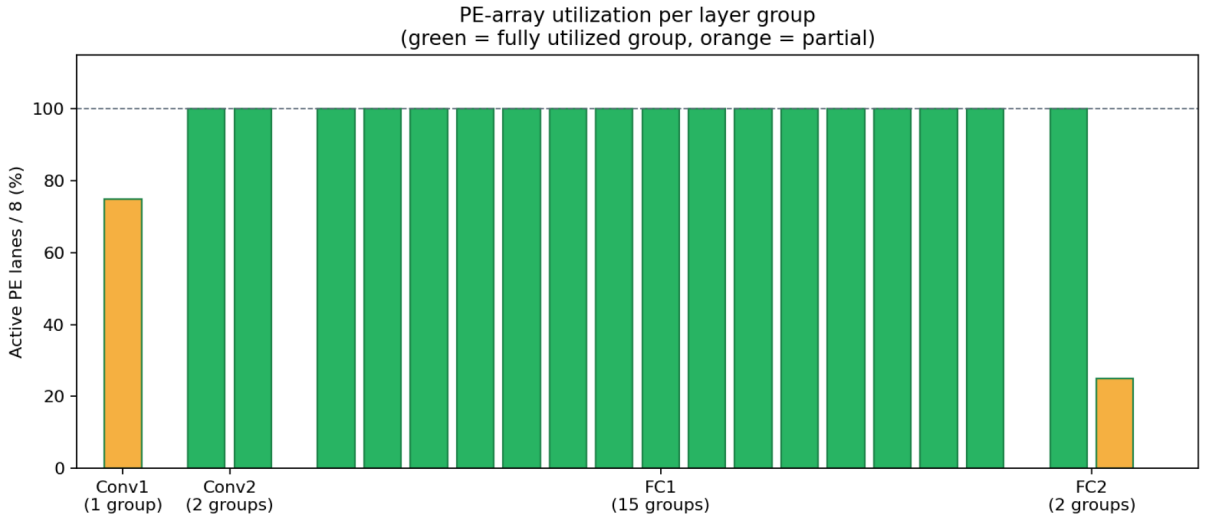
Pipelined pooling would shrink its share from 14.5% to a few percent and is listed as an improvement.

Choosing the number of lanes



Lanes per engine	Total lanes	Cycles	Latency at 150 MHz	Utilization	Cycles × lanes
2	8	123,290	821.9 µs	100.0%	986,320
4	16	75,050	500.3 µs	88.4%	1,200,800
8	32	43,730	291.5 µs	88.3%	1,399,360
16	64	35,410	236.1 µs	60.0%	2,266,240

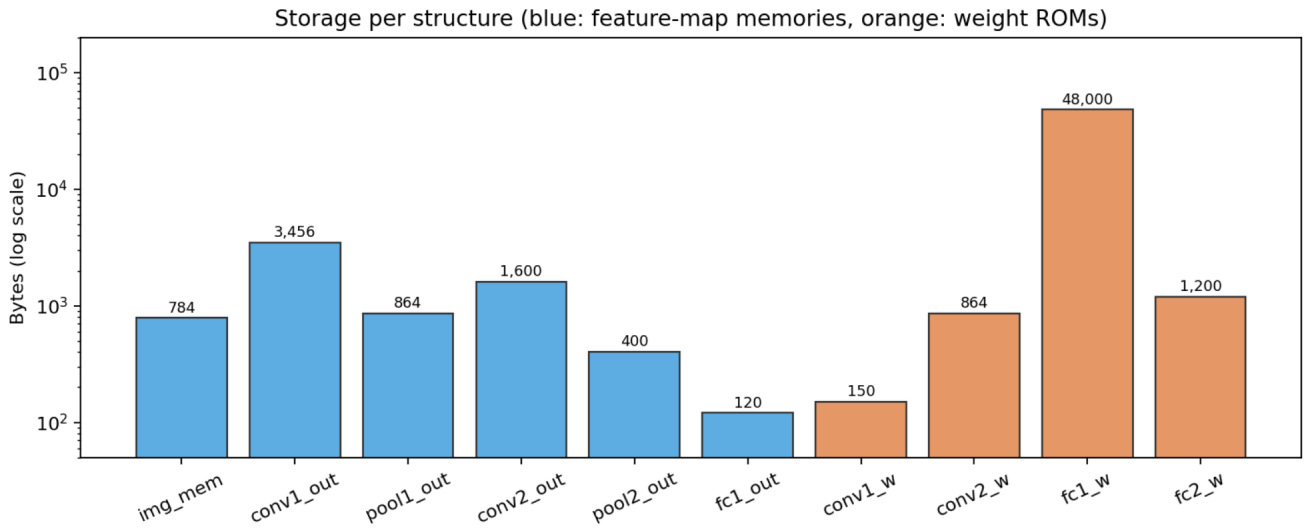
Why eight lanes



- 4 to 8 lanes: 1.72 times fewer cycles at 88% utilization
- 8 to 16 lanes: only 1.23 times, because Conv1 has 6 channels and FC2 has 10 outputs
- Conv2 and FC1 run every group at 100%
- Conv1 runs one group at 75% (6 of 8 lanes)
- FC2 runs a full group, then a group of 2 lanes, since 10 = 8 + 2
- Cycles times lanes favors 2 lanes, so latency was weighted: below 300 µs with utilization still high

$$\text{Utilization} = \frac{\text{active lanes in a group}}{N_{\text{PE}}} \times 100\%$$

Memory organization



- Logical storage totals 58,086 bytes: feature maps 7,224 B, weights 50,214 B, 152 biases as 32 bit words 608 B, score registers 40 B
- FC1 weights are 48,000 bytes, 96% of all weight storage and about 55 times the Conv2 ROM
- Physical footprint depends on how synthesis realizes the multi port ROMs

Storage design decisions

- Weight ROM, one read port per lane (weight_rom.v): every lane fetches its weight in the same cycle, so no weight fetch stage exists
- Weights preload from hex files with $readmemh
- On an FPGA the small convolution ROMs, at most 864 entries, map to per lane distributed ROM copies
- The 8 port FC1 ROM is the one to target first, for example by banking weights across lanes
- Single write port outputs: one channel is written per cycle after the tap loop, at most 8 extra cycles against 25 or 54 tap cycles
- Input FIFO of 1024 × 8: larger than the 784 pixel frame, so AXI writes finish immediately without flow control

Memory sizing mathematics

Memory	Depth	ADDR_W
img_mem, 28×28	784	10
conv1_out_mem, 6×24×24	3,456	12
pool1_out_mem, 6×12×12	864	10
conv2_out_mem, 16×10×10	1,600	11
pool2_out_mem, FC1 input	400	9
fc1_out_mem, FC2 input	120	7
Conv1 weights, 6 × 25	150	8
Conv2 weights, 16 × 54	864	10
FC1 weights, 120 × 400	48,000	16
FC2 weights, 10 × 120	1,200	11

$$\mathrm{ADDR_W} = \lceil \log_2 \mathrm{DEPTH} \rceil$$

- Each memory is sized to exactly its element count
- 7,224 + 50,214 + 608 + 40 = 58,086 bytes
- FIFO depth 2^{10} = 1024 is the smallest power of two holding 784 pixels

AXI4-Lite register map

Offset	Register	Access	Meaning
0x00	CTRL	W	Bit 0 is the start pulse, the remaining bits are reserved
0x04	STATUS	R	Bit 0 busy, bit 1 done, bit 2 input loaded
0x08	INPUT_DATA	W	Push the next input image byte
0x0C	PERF_CYCLES	R	Cycle count of the last run
0x10	PERF_MACS	R	Executed, non skipped MAC operations
0x14	PERF_SKIP	R	Zero gated MAC operations
0x40 + 4i	OUT_SCORE(i)	R	Signed INT32 score for class i, i = 0 to 9

Register offsets are defined once in cnn_pkg.vh and shared by the RTL and the testbench.

Why AXI4-Lite

- The accelerator's I/O is register shaped: a start command, a status word, one frame of input and a few result registers
- AXI4-Stream would add TVALID and TREADY back pressure handling that this workload does not benefit from
- Full AXI4 burst support would add address phase complexity for no gain, since the input is a single 784 byte frame written one byte per access
- Sizing the input FIFO to hold a whole frame means writes complete immediately with no back pressure, which lets a simple register style slave carry a streaming input

Problem statement capabilities and where they map

Capability	Register
Configure and start	CTRL
Provide input data	INPUT_DATA
Monitor status	STATUS
Read the output result	OUT_SCORE(i), PERF_*

Host programming flow

```
1. write INPUT_DATA <- pixel[0..783] (784 writes)
2. write CTRL <- 1 (start)
3. read STATUS until done == 1
4. read PERF_CYCLES, PERF_MACS, PERF_SKIP
5. read OUT_SCORE(0) .. OUT_SCORE(9)
```

- Back to back inference needs only a fresh start, with no reset in between
- Each write is a full 32 bit access and each pixel occupies the low 8 bits of INPUT_DATA
- The testbench implements exactly this flow in its AXI4-Lite bus functional model

Write channel FSM

- W_IDLE waits for AWVALID and WVALID together
- AWREADY and WREADY pulse for one cycle on acceptance
- The address decodes: CTRL sets a start pulse from wdata[0], INPUT_DATA raises a one cycle write strobe with wdata[7:0]
- Writes to other addresses are ignored and BRESP is OKAY
- W_RESP holds BVALID until BREADY

Registered outputs

Every strobe and response is registered. The start and input write strobes last exactly one axi_aclk cycle, which is what the pulse synchronizer and the FIFO write side expect.

Read channel FSM and compliance scope

- R_IDLE waits for ARVALID
- ARREADY pulses and the register value is captured
- STATUS returns input loaded, done and busy in bits 2, 1 and 0
- Unmapped addresses return 0xDEADBEEF
- R_DATA holds RVALID until RREADY, RRESP is OKAY

Compliance, stated precisely

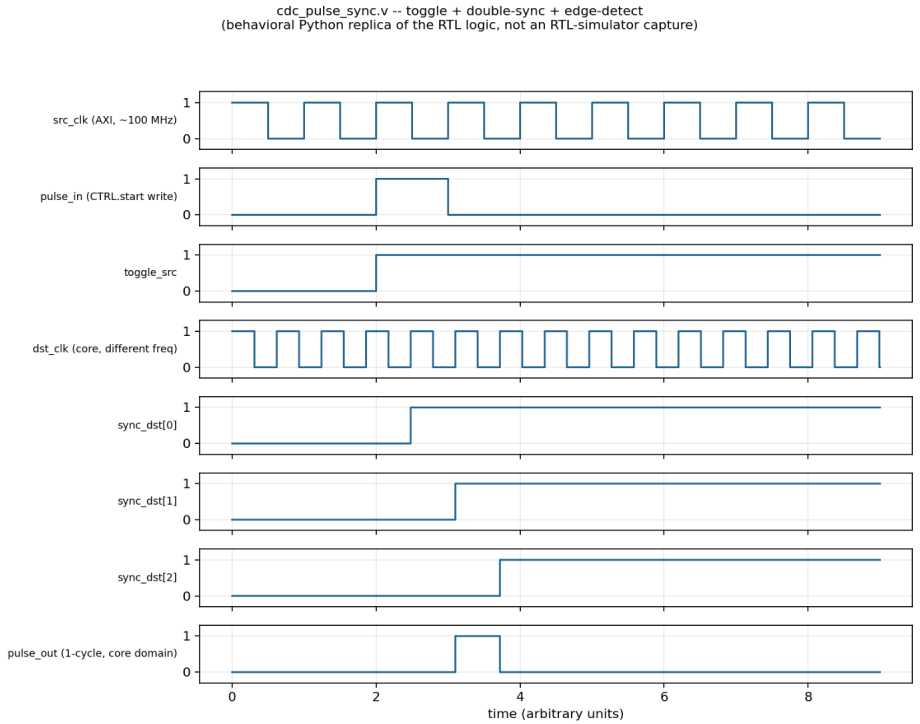
- AWVALID and WVALID are intentionally coupled
- WSTRB is accepted but not honored
- Not a fully AXI4-Lite compliant slave
- It serves a single master register access pattern
- FIFO full is unused, a streaming design would back pressure

Clock domain crossing inventory

Signal	Type	Mechanism	Reason
start	One cycle pulse, AXI to core	Toggle, two flop sync, edge detect	A one cycle pulse can be missed by an unrelated clock
busy, done, input_loaded	Slow levels, core to AXI	Two flop synchronizer	Levels persist, so plain synchronization is enough
Input pixels	Continuous 8 bit stream, AXI to core	Gray code asynchronous FIFO	Multi bit data must never be sampled mid transition
Scores, counters	Wide values, core to AXI	Read after done, stable before enable	Values are frozen before the qualifying flag is observed

Three different mechanisms, because three kinds of signal have three different failure modes.

Toggle based pulse synchronizer



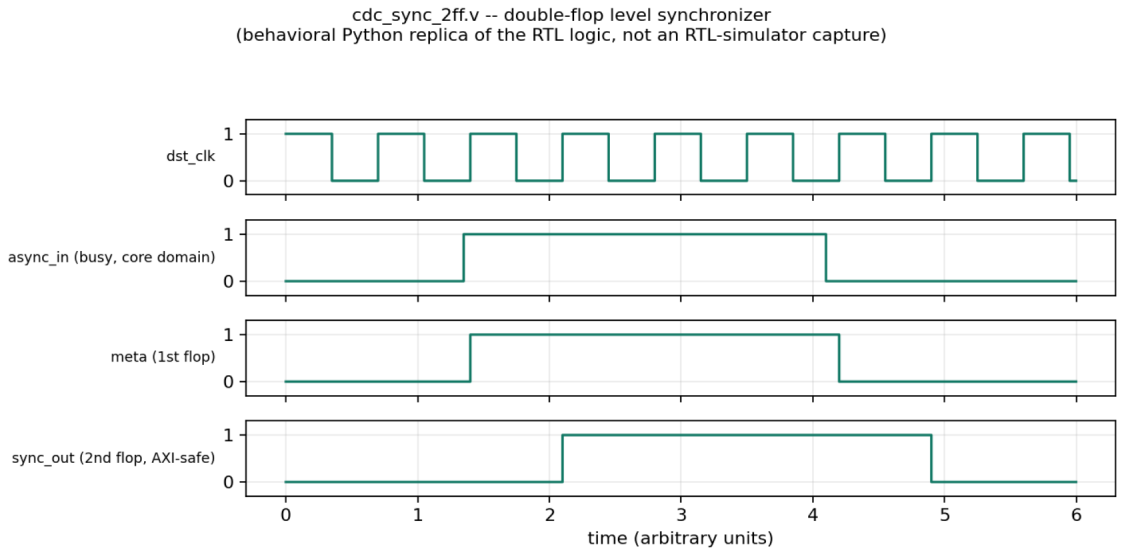
- A one cycle axi_aclk pulse is not guaranteed to be seen by a core clock that is faster, slower or unrelated
- The source domain toggles a level on every pulse
- The destination double flops the level and an edge detector rebuilds a one cycle pulse
- A toggle held for a full source period is always long enough to be caught
- A plain two flop synchronizer would be unsafe for this signal

Pulse synchronizer, step by step

Step	Domain	What happens
1	axi_aclk	ctrl_start_pulse is high for one cycle
2	axi_aclk	toggle_src inverts and then holds its level
3	core_clk	sync_dst[0] samples the level and may go metastable
4	core_clk	sync_dst[1] captures a resolved value
5	core_clk	sync_dst[2] takes the previous value of sync_dst[1]
6	core_clk	pulse_out = sync_dst[2] ^ sync_dst[1] is high for one cycle

- XOR means either edge of the toggle marks one pulse, so no clearing handshake is needed
- Latency is about three destination cycles
- Limit: two pulses closer than about three destination cycles can merge, acceptable because start is issued rarely

Two flop synchronizer for status levels



- busy, done and input_loaded change rarely compared with either clock
- They need only a bounded latency, metastability safe crossing
- Two destination flops give the first flop time to resolve before the value is used
- Latency is two destination cycles
- A pulse synchronizer here would be needless overhead

The two mechanisms are not interchangeable: each is used where it is correct.

Why an asynchronous FIFO for pixels

- The pixel path is a continuously changing multi bit bus
- A two flop synchronizer protects one bit only, so sampling several bits can capture different bits mid transition and corrupt the value
- async_fifo.v keeps the data in dual port memory and synchronizes only the read and write pointers
- Pointers are Gray coded, so only one pointer bit changes per increment and a sampled pointer is always the old or the new value
- Full and empty come from comparing synchronized pointers, which errs in the safe direction: a stale pointer can only make the FIFO look fuller or emptier, never lose or duplicate a word
- Depth 1024 × 8 holds a whole 784 pixel frame

Asynchronous FIFO pointer logic

Element	Implementation
Pointers	ADDR_W + 1 bit binary counters on each side, the extra MSB separates full from empty
Gray conversion	gray = (bin >> 1) ^ bin, registered next to the binary pointer
Storage	Dual port array indexed by the low ADDR_W bits, written on wr_clk and read on rd_clk
Crossing	Each Gray pointer passes through two flops in the opposite domain
Full	Next write pointer equals the synchronized read pointer with its top two bits inverted
Empty	Next read pointer equals the synchronized write pointer
Read data	Registered, so it appears one core cycle after the read enable

Scores and counters: stable before enable

- Performance counters and scores are wide buses read over AXI after the run
- Once the last layer engine finishes writing, they no longer change
- The done flag is double flopped, so by the time the AXI side sees done the data has been static for at least the synchronizer latency
- Safety comes from the values being quiet before the qualifying flag is observed, a bundled data argument
- It does not come from any claim that a synchronizer can carry a changing bus

Why no synchronizer on the result buses

A two flop synchronizer on a many bit bus that changes several bits per cycle would not synchronize it safely, since different bits could resolve on different cycles and corrupt the value.

Input loader and FIFO read alignment

Aligning to read latency

The FIFO returns data one cycle after rd_en, so the loader delays the write enable and address by one cycle (fifo_rd_en_d, img_load_addr_d).

Shared image memory port

While the delayed write enable is high the loader owns the img_mem address, otherwise Conv1 does. Only one side is active at a time.

Leaving the LOAD state

fifo_rd_en needs a non empty FIFO, a count below 784 and the LOAD state. Conv1 starts when input_loaded is true and no read is in flight.

Counter reset on every start

The loader counter clears on each start pulse. Gating it on IDLE left it at 784 after run one, because DONE returns to LOAD without passing IDLE.

Zero gating with operand isolation

```
wire is_zero_act = (act_in == 0);
wire do_mac = en & ~is_zero_act;
wire gated_act = do_mac ? act_in : 0;
wire gated_wgt = do_mac ? wgt_in : 0;
wire product = gated_act * gated_wgt;
```

- Each lane checks whether its activation is exactly zero. If so, the multiply and the accumulator update are both suppressed and the accumulator holds
- Forcing both multiplier inputs to a constant zero stops the multiplier's internal nodes from toggling with act_in and wgt_in
- Merely skipping the accumulator update would leave the multiplier switching every cycle, so isolation makes the saving real
- The gate sits inside every mac_unit, so Conv1, Conv2, FC1 and FC2 are covered with no special cases

Where, how, benefit and overhead

Where

Inside every mac_unit, at the activation input of each lane, in all four layer engines.

How

An exact zero activation forces both multiplier operands to zero and holds the accumulator. A skip flag is raised and counted.

Expected benefit

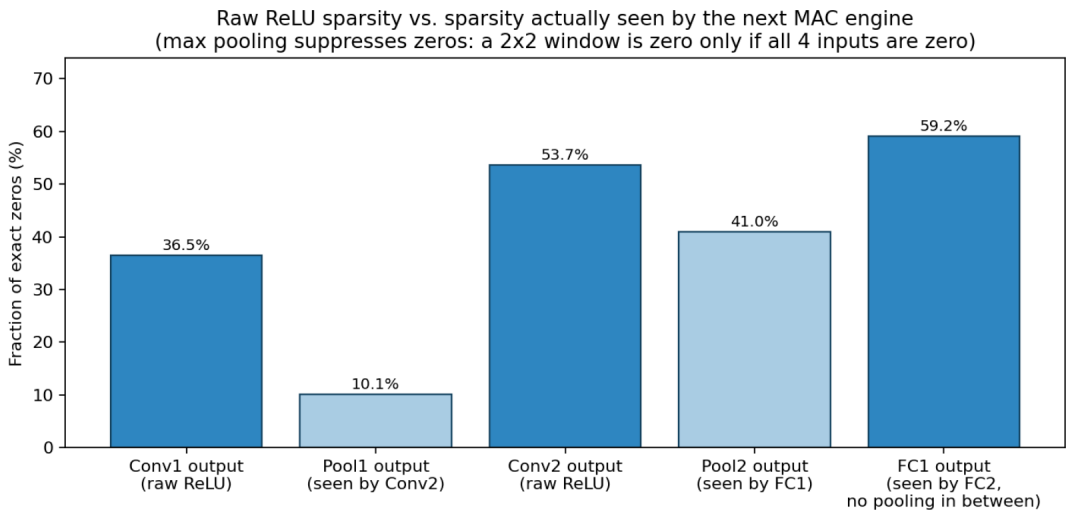
Lower dynamic power from reduced multiplier and accumulator switching. Latency is unchanged, because the schedule is fixed.

Overhead

One 8 input zero detect, two 8 bit isolation gates and one skip flag per lane, plus the performance counters.

`PERF_CYCLES` is independent of sparsity; `PERF_MACS` and `PERF_SKIP` are functions of it

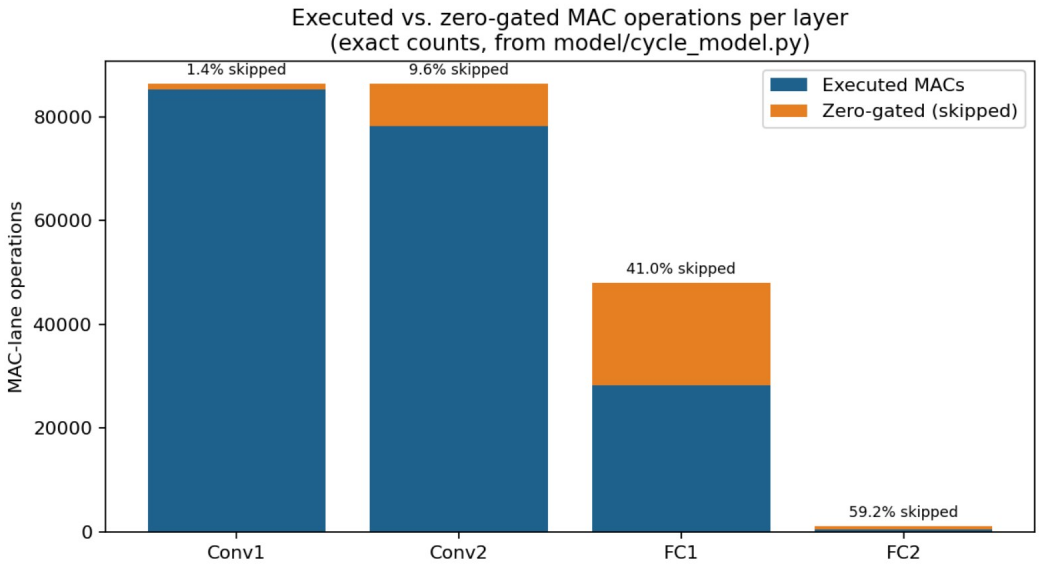
What each layer sees



Transition	Raw zeros	Gated view
Conv1 to Conv2, via Pool1	36.5%	9.6%
Conv2 to FC1, via Pool2	53.7%	41.0%
FC1 to FC2	59.2%	59.2%

- A 2×2 max window is zero only if all four inputs are zero, which under independence gives p^4
- Real activations are spatially correlated, so the measured values are reported
- FC layers re read each input equally often, so the skip fraction equals the raw zero fraction exactly

Where the savings land



Layer	Executed	Gated	Share
Conv1	85,230	1,170	1.35%
Conv2	78,144	8,256	9.56%
FC1	28,320	19,680	41.00%
FC2	490	710	59.17%
Total	192,184	29,816	13.43%

$$\text{Sparsity} = \frac{N_{\text{skip}}}{N_{\text{skip}} + N_{\text{mac}}} = \frac{29{,}816}{222{,}000} \approx 13.4\%$$

Conv1 sees the raw image, which has almost no exact zeros. The benefit grows deeper in the network.

Why gate on activations only

- Considered: if (act == 0 || wgt == 0)
- Weight sparsity is a static property of the trained model and needs a separate compression or scheduling story to exploit properly
- Activation zeros are dynamic, arise from one clear cause (ReLU) and are detected with the same eight input zero check
- One cause and one detection point make the optimization easy to explain and to verify
- The activation function and the power optimization form one coherent story

Next step, not taken

Skipping whole zero taps in the controller would turn the switching saving into a latency saving. FC1 and FC2 would benefit most, but the schedule would become data dependent.

Performance counters and the event invariant

Counter	Increments by
PERF_CYCLES	1 on every cycle in which busy is high
PERF_MACS	Population count of lanes with mac_valid
PERF_SKIP	Population count of lanes with mac_skipped

$$6 \cdot 576 \cdot 25 \;+\; 2 \cdot 8 \cdot 100 \cdot 54 \;+\; 15 \cdot 8 \cdot 400 \;+\; (8{+}2) \cdot 120 \;=\; 222{,}000$$

- An early version OR reduced the lane vectors and counted one event per cycle, undercounting when several lanes fired together. Each engine's vector is now popcounted and the sums added
- The total is fixed by the schedule alone, so executed plus skipped must equal 222,000 for any data

Bit exact golden model

- model/golden_model.py is a NumPy forward pass that uses the same integer arithmetic as the RTL
- Same accumulation, same ReLU then shift then saturate sequence, same channel major flatten: idx = ch × 25 + row × 5 + col
- All tensors use channel, height, width order, so every RTL memory address lines up with the model
- It writes the weight, bias and image hex files and the expected scores that the testbench compares against

Reference scores of the verification workload, classes 0 to 9

418, 1341, 1628, −4045, −815, −2557, 3165, −902, −498, 193. The argmax, class 6, is a functional test result. The weights are fixed seed synthetic values, so it is not an accuracy measurement.

Self checking testbench

Test	What it does	What it catches
1. Normal inference	Streams 784 bytes, starts, polls done, reads PERF_* and 10 scores, compares bit for bit with the golden model	Datapath, dataflow, addressing and quantization errors
2. Repeated inference	Runs the sequence again with no reset, only a fresh start, and compares with the golden model and run 1	State that fails to reinitialize, including the loader counter across DONE to LOAD
3. Counter invariant	Checks PERF_MACS + PERF_SKIP equals exactly 222,000	Errors in the instrumentation, independent of data

- An AXI4-Lite bus functional model drives the DUT with a 100 MHz AXI clock and an unrelated core clock of about 150 MHz, exercising every CDC block
- Watchdogs name the stalled handshake after 200 cycles per access, 5,000 status polls, or a 4,000,000 ns global limit

Repository, pipeline and trained model swap

Repository layout

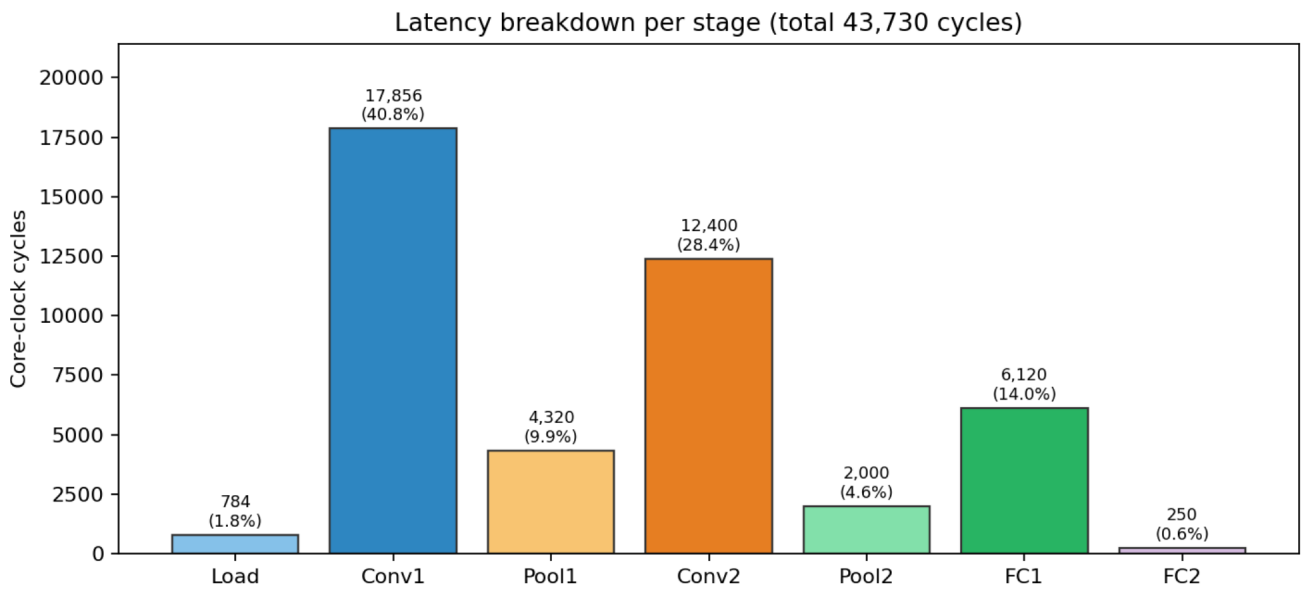
- rtl synthesizable Verilog, one module per file
- tb testbench and generated vectors
- model golden model, cycle model, PE sweep, figures
- sim run_sim.sh and run_sim.bat
- docs figures used in the README

One command pipeline

- Regenerate weight, bias, image and expected score files
- Compile RTL and testbench with Icarus Verilog
- Simulate and write rtl_results.txt
- Patch README section 13.3 with the counters
- Regenerate every figure

Trained model swap: only get_weights_and_image() in weights_source.py changes. Update the SHIFT constants in config.py and cnn_pkg.vh if per layer scales differ. No RTL change is needed.

Modeled latency breakdown



- 43,730 cycles per inference
- 291.5 µs at 150 MHz
- About 3,430 inferences per second
- Conv1 40.8%, Conv2 28.4%, FC1 14.0%
- Pooling 14.5%, load 1.8%

Scope of these numbers

Cycle counts come from the deterministic schedule model, and the 150 MHz core clock is a testbench assumption. The Conv1 engine measured 17,856 cycles in RTL simulation, matching the model. No synthesis was run, so LUT, FF, BRAM and Fmax are not reported.

Latency and efficiency analysis

Stage	Cycles	Share	Executed	Gated
Input load	784	1.8%		
Conv1	17,856	40.8%	85,230	1,170
Pool1	4,320	9.9%		
Conv2	12,400	28.4%	78,144	8,256
Pool2	2,000	4.6%		
FC1	6,120	14.0%	28,320	19,680
FC2	250	0.6%	490	710
Total	43,730	100%	192,184	29,816

- Peak rate: 8 lanes × 150 MHz = 1.2 GMAC/s
- Sustained: 192,184 / 291.5 µs ≈ 0.66 GMAC/s
- Lane utilization during tap cycles: 88.3%
- Convolution takes 69.2% of cycles, pooling 14.5%
- Pooling is the largest remaining optimization target

$$\text{Latency} = \frac{43{,}730}{150\ \text{MHz}} \approx 291.5\ \mu\text{s}, \qquad \text{Throughput} \approx 3{,}430\ \text{inferences per second}$$

Design trade offs

Decision	Chosen over alternatives	Reason
Precision	INT8 with INT32 accumulate, over FP32 or INT16	Four times less storage than FP32, small multipliers, no overflow risk
Dataflow	Output stationary, over weight or input stationary	Local partial sums and minimal partial sum traffic
Lanes	8 per engine, over 4 or 16	1.72 times faster than 4, while 16 gives 1.23 times at 60% utilization
Engines	Four engines, over one shared array	Independent layers and simple control, at the cost of 32 lanes
Interface	AXI4-Lite, over AXI4 or AXI4-Stream	Register shaped I/O needs no bursts or back pressure
CDC	Three matched mechanisms, over one generic synchronizer	Each signal class has its own failure mode
Optimization	Activation gating, over weight gating or compaction	One cause, small overhead, directly measurable
Scheduling	Sequential layers, over overlap	Deterministic latency and simpler memory and control

Current limitations

- Weights are fixed seed INT8 values, not a trained classifier, and are loaded at elaboration
- No synthesis or place and route was run, so LUT, FF, BRAM, Fmax and power are not claimed
- The 8 port FC1 ROM needs banking before it maps well to a device
- Zero gating reduces switching activity, not latency
- Four engines hold 32 physical lanes where one shared array needs 8
- The AXI slave couples AW and W, ignores WSTRB and serves a single master

Planned improvements, in order of impact

- Compress the schedule around zero activations, turning switching savings into latency savings
- Time multiplexed shared array: one pe_array with a mode input would cut lanes from 32 to 8
- Runtime configurable lane count: a control register selecting 2, 4, 8 or 16 lanes
- Pipelined pooling: overlapped window fetches to shrink the 14.5% share
- Weight loading over AXI and banking the 48,000 entry FC1 ROM by lane
- Independent AXI4-Lite channels for multi master use
- Trained model evaluation through the single file change in weights_source.py
- Power and physical metrics: switching activity analysis and place and route for Fmax and utilization